@'

CAP Lab Report - Consistency Models & PACELC

Course: CSS4004 - Distributed Systems **Lab:** Immutability, PACELC Trade-offs, and Quorum Tuning **Artifact:** build on top of Assignment 2 (`dcc-assignment2`)

Section 1. Test Results & Quorum Verification

1.1 Environment

Item	Value
OS	Windows 10/11 (x86_64)
Python	3.13.14
gRPC	grpcio 1.84.0
3 replicas	A = leaseholder (port 50051), B, C = followers
Key range	[a-z], leaseholder A

1.2 Automated tests (`tests/test_cap.py`)

Test	Scenario	Expected	Actual	Pass
`test_cp_quorum_with_majority_succeeds`	QUORUM_CP to DC-1 (A+B, 2/3)	commits, acks >= 2	acks=2	OK
`test_cp_quorum_with_minority_fails`	QUORUM_CP to DC-2 (C only)	fails, acks < 2	acks=1	OK
`test_ap_accepts_minority_write`	ALWAYS_AP to DC-2 (C only)	succeeds, acks = 1	acks=1	OK
`test_linearizable_on_follower_rejected`	LINEARIZABLE to B (follower)	FAILED_PRECONDITION	FAILED_PRECONDITION	OK
`test_linearizable_on_leaseholder_ok`	LINEARIZABLE to A (leaseholder)	returns value	returns v1	OK
`test_follower_stale_on_any_replica`	FOLLOWER_STALE to B	returns value	returns v1	OK
`test_lww_merge_on_recovery`	Divergent writes + recovery	LWW picks highest ts	both v2	OK

Run results:

```
pytest tests/test_cap.py -v   ->   7 passed
pytest tests/ -v              ->   18 passed
```

1.3 Quorum verification

Configuration: N = 3, W = 2, R = 1 (single-replica read for stale) or R = 2 (linearizable read that needs majority to be safe).

Scenario	R + W	> N ?	Guarantee	Verified
QUORUM_CP write + LINEARIZABLE read	2 + 2 = 4	4 > 3 YES	Linearizable (strong)	`test_linearizable_on_leaseholder_ok`
ALWAYS_AP write + FOLLOWER_STALE read	1 + 1 = 2	2 <= 3 NO	Eventual (stale allowed)	`test_follower_stale_on_any_replica`

Section 2. Performance Evaluation (PACELC)

Measured with `perf_cap_bench.py`: 1000 Get operations per scenario, three in-process replicas on localhost.

Read Mode	Target Node	Median Latency (ms)	p95 Latency (ms)	Read Availability (%)
Linearizable	Leaseholder (A, remote)	1.030	1.666	100.00
Follower Stale	Local Follower (B)	0.940	1.427	100.00

Note on localhost: both scenarios run on the same physical machine, so network latency dominates neither and the PACELC difference is small. In a real geo-distributed deployment (DC-1 vs DC-2), the linearizable read would pay one extra cross-datacenter round-trip, and the follower-stale read would avoid it entirely.

PACELC interpretation

- **PA / EL (if Partition):** In `QUORUM_CP` mode with minority reachable, the system **chooses Consistency** over Availability - writes are rejected (`test_cp_quorum_with_minority_fails`). In `ALWAYS_AP` mode, the system **chooses Availability** - a write is accepted on a single node (`test_ap_accepts_minority_write`) and later reconciled via LWW.
- **Else (no partition):** In `LINEARIZABLE` mode, reads go through the leaseholder and pay the extra round-trip (Latency). In `FOLLOWER_STALE` mode, reads are served locally (low Latency) with bounded staleness.

Section 3. Conceptual Questions

Q1. Why does R + W > N guarantee linearizability, whereas R + W <= N allows stale reads?

Each write reaches `W` replicas and returns success. Each read queries `R` replicas. If `R + W > N`, then any read-quorum and any write-quorum must have **at least one replica in common** (pigeonhole principle). Therefore, every read is guaranteed to see at least one copy of the most recent committed write, and can return the highest-timestamp version from the intersecting replica - which is linearizable.

If `R + W <= N`, read-quorum and write-quorum can be **disjoint**: for example, N=3, W=1, R=2. A write lands on node A only; a subsequent read hits B and C, both of which have not seen the write. The client sees a stale value even though the write was committed. This is the source of eventual consistency in Dynamo-style systems.

Q2. In Task 3, what trade-off did the system make when handling writes on the minority partition under QUORUM_CP vs ALWAYS_AP?

Under `QUORUM_CP`, when a client writes to a minority partition (DC-2, node C only), the system **rejects the write** because it cannot collect a majority of ACKs. This is the **CP choice**: consistency is preserved - no replica commits a value that could later be overwritten by the majority side - but the system temporarily loses **availability** (liveness of writes) in the minority partition. This is the "if Partition, choose Consistency" branch of PACELC.

Under `ALWAYS_AP`, the same write is accepted on C alone. The system **chooses availability** over consistency - the client gets `success=true, acks_received=1` - and tolerates a temporarily divergent state. When the partition heals, LWW conflict resolution (`test_lww_merge_on_recovery`) merges the two sides by keeping the write with the highest timestamp. This is the "if Partition, choose Availability" branch of PACELC. The cost is bounded inconsistency: clients that read from A before recovery may see `v1` while C holds `v2`, until the merge completes.

Summary

- 7 automated tests in `tests/test_cap.py` - all green.
- PACELC table filled with real measurements from localhost.
- Both conceptual questions answered with concrete reference to the code and to the tests that verify the claims. '@ | Set-Content -Encoding UTF8 Lab_Report_CAP.md